Pattern Matching: *variant-like* and `std::expected`

Document #:	P3527R0
Date:	2024-12-17
Project:	Programming Language C++
Audience:	Evolution Library Evolution

Reply-to:	Michael Park <mcypark@gmail.com> Zach Laine <whatwasthataddress@gmail.com>

Contents

§ 1 Motivation and Scope

[P2688R3] introduces pattern matching for C++, including handling of `std::variants`. For example:

```
auto v = std::variant<int32_t, int64_t, float, double>{/* ... */};
v match {
  int32_t: let i32 =>
    std::print("got int32: {}", i32);
  int64_t: let i64 =>
    std::print("got int64: {}", i64);
  float: let f =>
    std::print("got float: {}", f);
  double: let d =>
    std::print("got double: {}", d);
};
```

However, there is an unsupported type in the standard that is variant-like but is not covered by [P2688R3] without explicit library support. That type is `std::expected`.

This proposal does not interact with the pattern matching language feature, nor does it interact with user defined types; it only affects whether `std::expected` interoperates gracefully with the language feature.

This paper adds library facilities that allow `std::expected` to conform to the notion of variant-like used in the pattern matching core wording. In order to accomplish this, we propose to do three things:

1. Introduce the *variant-like* exposition-only concept.
2. Add `std::expected` to the *variant-like* concept.
3. Modify `std::visit` to handle *variant-like* types (optional).

This has the side effect that users will now be able to use `std::visit` in a generic context to visit alternatives of both `std::variant` (as is the status quo) and `std::expected` (via the changes in this paper).

§ 1.1 The Variant Protocol

The language semantics for pattern matching ([P2688R3]) include the notion of a "variant protocol". This is analogous to the tuple protocol used for structured bindings. The tuple protocol is used in defining the behavior of pattern matching as well.

This section does not describe a library change, and is not part of this proposal. It is here to describe the relevant language part of the pattern matching design, since that language design informs the library design in this paper.

§ 2 Design

There are two options for exactly how to define the *variant-like* concept.

§ 2.1 Option 1: An Explicit List of Enabled Templates

This is what the *tuple-like* concept does. The standard says, "A type `T` models and satisfies the exposition-only concept *tuple-like* if `remove_cvref_t<T>` is a specialization of `array`, `complex`, `pair`, `tuple`, or `ranges::subrange`."

So, Option 1 would be to say nearly the same thing: "A type `T` models and satisfies the exposition-only concept *variant-like* if `remove_cvref_t<T>` is a specialization of `expected` or `variant`."

This has the benefit of being consistent with how *tuple-like* is defined.

It has the disadvantage of being a maintenance burden. Future library additions of `variant`-like templates must remember to update the wording for *variant-like*.

§ 2.2 Option 2: A More Complicated, More General Concept

Another way to define *variant-like* would be to provide wording that applies to `std::variant` and `std::expected`, which is designed to be general enough to encompass similar types.

This has the benefit of being more maintainable.

This definition also has the disadvantage of novelty, and specifically of not having more than two templates that model it. This means that it might not work just to add new templates without adjusting *variant-like* anyway, for reasons we can not anticipate now.

§ 2.3 Modifying `std::visit`

Modifying `std::visit` to handle *variant-like* types is optional. It seems useful to be able to use `std::visit` for `std::expecteds`, since they are gaining *variant-like* status. However, modifying `std::visit` will not be directly used by the pattern matching language rules. In fact, once pattern matching is available, the use cases for `std::visit` will be far rarer. This addition is something that LEWG should poll separately from the rest of the paper.

The idea of the modification is to change the names of all the exposition-only *as-variant* overload set described in 22.6.7 [variant.visit] to *as-variant-like*, change `std::visit` to use *as-variant-like*, then add these four new overloads:

```
template<class T, class E>
  constexpr auto&& as-variant-like(expected<T, E>& var) { return var; }
template<class T, class E>
   constexpr auto&& as-variant-like(const expected<T, E>& var) { return
        var; }
template<class T, class E>
    constexpr auto&& as-variant-like(expected<T, E>&& var) { return
        std::move(var); }
```

```
template<class T, class E>
   constexpr auto&& as-variant-like(const expected<T, E>&& var) { return
        std::move(var); }
```

We would also need to add overloads of the *GET* exposition-only function defined in 22.6.5 [variant.get]:

```
template<size_t I, class... Types>
  constexpr variant_alternative_t<I, variant<T, E>>&
    GET(expected<T, E>& v);
template<size_t I, class... Types>
  constexpr variant_alternative_t<I, variant<T, E>>&&
    GET(expected<T, E>&& v);
template<size_t I, class... Types>
  constexpr const variant_alternative_t<I, variant<T, E>>&
    GET(const expected<T, E>& v);
template<size_t I, class... Types>
  constexpr const variant_alternative_t<I, variant<T, E>>&&
    GET(const expected<T, E>&& v);
```

§ 3 Proposed Wording

Add a new section [variant.like] after 22.6.2 [variant.syn]:

§ 22.6.2+1 [variant.like] Concept *variant-like*

```
+ template<class T>
+   concept variant-like = see below;           // exposition only
```

Then, define *variant-like* as outlined in Option 1:

1 A type `T` models and satisfies the exposition-only concept *variant-like* if `remove_cvref_t<T>` is a specialization of `expected` or `variant`.

… or define *variant-like* as outlined in Option 2:

1 A type `T` models and satisfies the exposition-only concept *variant-like* if `remove_cvref_t<T>` is a specialization of `expected` or `variant`.

Add a new member function `index` to 22.8.6.1 [expected.object.general]

§ 22.8.6.1 [expected.object.general] General

```
namespace std {
  template<class T, class E>
  class expected {
  public:
    ...

    // [expected.object.obs], observers
    ...
    template<class U> constexpr T value_or(U&&) const &;
```

```
    template<class U> constexpr T value_or(U&&) &&;
    template<class G = E> constexpr E error_or(G&&) const &;
    template<class G = E> constexpr E error_or(G&&) &&;
+   constexpr size_t index() const noexcept;

    ...
  };
}
```

Add a new member function `index` to 22.8.6.6 [expected.object.obs]

§ 22.8.6.6 [expected.object.obs] Observers

```
+ constexpr size_t index() const noexcept;
```

26 *Returns*: `has_value() ? 0 : 1`.

Add a new member function `index` to 22.8.7.1 [expected.void.general]

§ 22.8.7.1 [expected.void.general] General

```
template<class T, class E> requires is_void_v<T>
class expected {
public:
  ...

  // [expected.void.obs], observers
  ...
  template<class G = E> constexpr E error_or(G&&) const &;
  template<class G = E> constexpr E error_or(G&&) &&;
+ constexpr size_t index() const noexcept;

  ...
};
```

Add a new member function `index` to 22.8.7.6 [expected.void.obs]

§ 22.8.7.6 [expected.void.obs] Observers

```
+ constexpr size_t index() const noexcept;
```

15 *Returns*: `has_value() ? 0 : 1`.

Add a new section [expected.asvariant] after 22.8.7.8 [expected.void.eq]:

§ 22.8.7.8+1 [expected.asvariant] Variant-like access to expected

```
+ template<class T, class E>
+    struct variant_size<expected<T, E>> : integral_constant<size_t, 2> {
        };
+
+ template<size_t I, class T, class E>
+   struct variant_alternative<I, expected<T, E>> {
```

```
+     using type = see below ;
+   };
```

1 *Mandates*: `I < 2`

2 *Type*: The type `T` if `I` is `0`, otherwise the type `E`.

```
+ template<size_t I, class T, class E>
+    constexpr variant_alternative_t<I, expected<T, E>>& get(expected<T,
       E>& e);
+ template<size_t I, class T, class E>
+    constexpr variant_alternative_t<I, expected<T, E>>&& get(expected<T,
       E>&& e);
+ template<size_t I, class T, class E>
+    constexpr const variant_alternative_t<I, expected<T, E>>& get(const
       expected<T, E>& e);
+ template<size_t I, class T, class E>
+    constexpr const variant_alternative_t<I, expected<T, E>>&& get(const
       expected<T, E>&& e);
```

3 *Mandates*: `I < 2`.

4 *Throws*: `bad_variant_access` if `e.index() != I`.

— If `e.index()` is not `I`, throw an exception of type `bad_variant_access`.

— Otherwise, return `*e` if `has_value()` is `true`, and `e.error()` otherwise.

§ 3.0.1 In 22.6.5 [variant.get] add expected overloads of the *GET* exposition-only function.

```
template<size_t I, class... Types>
  constexpr variant_alternative_t<I, variant<Types...>>&
    GET(variant<Types...>& v);                                    //
       exposition only
template<size_t I, class... Types>
  constexpr variant_alternative_t<I, variant<Types...>>&&
    GET(variant<Types...>&& v);                                   //
       exposition only
template<size_t I, class... Types>
  constexpr const variant_alternative_t<I, variant<Types...>>&
    GET(const variant<Types...>& v);                              //
       exposition only
template<size_t I, class... Types>
  constexpr const variant_alternative_t<I, variant<Types...>>&&
    GET(const variant<Types...>&& v);                             //
       exposition only
```

3 *Mandates*: `I < sizeof...(Types)`.

4 *Preconditions*: `v.index()` is `I`.

5 *Returns*: A reference to the object stored in the `variant`.

```
template<size_t I, class T, class E>
  constexpr variant_alternative_t<I, expected<T, E>>&
```

```
    GET(expected<T, E>& v);                                          //
       exposition only
template<size_t I, class T, class E>
  constexpr variant_alternative_t<I, expected<T, E>>&&
    GET(expected<T, E>&& v);                                         //
       exposition only
template<size_t I, class T, class E>
  constexpr const variant_alternative_t<I, expected<T, E>>&
    GET(const expected<T, E>& v);                                    //
       exposition only
template<size_t I, class T, class E>
  constexpr const variant_alternative_t<I, expected<T, E>>&&
    GET(const expected<T, E>&& v);                                   //
       exposition only
```

3 *Mandates*: `I < 2`.

4 *Preconditions*: `v.index()` is `I`.

5 *Returns*: A reference to the object stored in the `expected`.

§ 3.0.2 In 22.6.7 [variant.visit] Change the name of the exposition-only *as-variant* to *as-variant-like*, and add overloads for `expected`.

1 Let ~~*as-variant*~~*as-variant-like* denote the following exposition-only function templates:

```
- template<class... Ts>
-   constexpr auto&& as-variant(variant<Ts...>& var) { return var; }
- template<class... Ts>
-   constexpr auto&& as-variant(const variant<Ts...>& var) { return var; }
- template<class... Ts>
-       constexpr  auto&&  as-variant(variant<Ts...>&&  var)  {  return
       std::move(var); }
- template<class... Ts>
-     constexpr auto&& as-variant(const variant<Ts...>&& var) { return
       std::move(var); }
+ template<class... Ts>
+   constexpr auto&& as-variant-like(variant<Ts...>& var) { return var; }
+ template<class... Ts>
+    constexpr auto&& as-variant-like(const variant<Ts...>& var) { return
       var; }
+ template<class... Ts>
+     constexpr auto&& as-variant-like(variant<Ts...>&& var) { return
       std::move(var); }
+ template<class... Ts>
+    constexpr auto&& as-variant-like(const variant<Ts...>&& var) { return
       std::move(var); }
+ template<class T, class E>
+   constexpr auto&& as-variant-like(expected<T, E>& var) { return var; }
+ template<class T, class E>
+    constexpr auto&& as-variant-like(const expected<T, E>& var) { return
       var; }
+ template<class T, class E>
+     constexpr auto&& as-variant-like(expected<T, E>&& var) { return
       std::move(var); }
```

```
+ template<class T, class E>
+   constexpr auto&& as-variant-like(const expected<T, E>&& var) { return
      std::move(var); }
```

Let *n* be `sizeof...(Variants)`. For each `0` ≤ *i* < *n*, let `V`$_i$ denote the type `decltype(`~~*as-variant*~~*as-variant-like*`(std::forward<Variants`$_i$`>(vars`$_i$`)))`.

2 Constraints: `V`$_i$ is a valid type for all `0` ≤ *i* < *n*.

3 Let `V` denote the pack of types `V`$_i$.

4 Let *m* be a pack of *n* values of type `size_t`. Such a pack is valid if 0 ≤ m_i < `variant_size_v<remove_reference_t<V`$_i$`>>` for all 0 ≤ *i* < *n*. For each valid pack *m*, let *e*(*m*) denote the expression:

```
INVOKE(std::forward<Visitor>(vis), GET<m>(std::forward<V>(vars))...)   //
see [func.require]
```

for the first form and

```
INVOKE<R>(std::forward<Visitor>(vis),    GET<m>(std::forward<V>(vars))...)
// see [func.require]
```

for the second form.

5 *Mandates*: For each valid pack *m*, *e*(*m*) is a valid expression. All such expressions are of the same type and value category.

6 *Returns*: *e*(*m*), where *m* is the pack for which m_i is ~~*as-variant*~~*as-variant-like*`(vars`$_i$`).index()` for all `0` ≤ *i* < *n*. The return type is `decltype(`*e*(*m*)`)` for the first form.

7 *Throws*: `bad_variant_access` if (~~*as-variant*~~*as-variant-like*`(vars).valueless_by_exception() || ...)` is `true`.

§ 4 Design Alternatives

§ 5 Implementation Experience

§ 6 References

[P2688R3] Michael Park. 2024-10-16. Pattern Matching: `match` Expression.
https://wg21.link/p2688r3